

Một vài nghiên cứu về danh sách liên kết theo khối

Su Yu

Trường Trung học trực thuộc Đại học Sơn Tây

Trại đông Tin học toàn quốc, 2008

Nguồn gốc: Một vài nghiên cứu về danh sách liên kết theo khối

IOI China candidate team papers / 2008 / Day 2 / Su Yu / block linked list study.doc

Tóm tắt

Bài viết giới thiệu khái niệm danh sách liên kết theo khối, cách mở rộng cấu trúc này, phân tích hiệu năng và bàn về mặt lợi hại khi dùng nó trong các kỳ thi tin học. Phần cuối nêu thêm một vài ứng dụng thực tế của tư tưởng chia khối.

Từ khóa: danh sách liên kết theo khối; kích thước khối; hiệu năng; mở rộng cấu trúc dữ liệu; mô phỏng; lấy điểm từng phần.

1 Danh sách liên kết theo khối là gì?

Ta bắt đầu từ một bài toán để thấy cấu trúc này xuất hiện tự nhiên.

1.1 NOI 2003: Editor

Bài toán yêu cầu xây một trình soạn thảo văn bản rỗng ban đầu. Văn bản là một dãy gồm các ký tự ASCII nhìn thấy được, con trỏ có thể nằm ở đầu, cuối hoặc giữa hai ký tự. Chương trình phải xử lý sáu thao tác sau.

Thao tác	Dạng trong input	Ý nghĩa
MOVE(<i>k</i>)	Move <i>k</i>	đưa con trỏ tới sau ký tự thứ <i>k</i>
INSERT(<i>n</i> , <i>s</i>)	Insert <i>n</i> rồi chuỗi <i>s</i>	chèn <i>n</i> ký tự tại con trỏ
DELETE(<i>n</i>)	Delete <i>n</i>	xóa <i>n</i> ký tự sau con trỏ
GET(<i>n</i>)	Get <i>n</i>	xuất <i>n</i> ký tự sau con trỏ
PREV()	Prev	lùi con trỏ một ký tự
NEXT()	Next	tiến con trỏ một ký tự

Ví dụ, từ trình soạn thảo rỗng, lần lượt thực hiện INSERT(13, "Balanced tree"), MOVE(2), DELETE(5), NEXT(), INSERT(7, " editor"), MOVE(0), GET(16) thì kết quả là "Bad editor tree".

Giới hạn chính của bài là: số thao tác MOVE không quá 50000, tổng số thao tác INSERT và DELETE không quá 4000, tổng số PREV và NEXT không quá 200000. Tổng số ký tự được chèn không vượt quá 2 MB, còn output đúng không vượt quá 3 MB. Mọi thao tác đều hợp lệ.

Thực chất các lệnh chỉ thuộc hai nhóm: định vị, và thêm hoặc xóa. Hai cách lưu trữ tuần tự quen thuộc có ưu nhược điểm đối lập:

	mảng	danh sách liên kết
định vị	$\mathcal{O}(1)$	$\mathcal{O}(N)$
thêm hoặc xóa	$\mathcal{O}(N)$	$\mathcal{O}(1)$

Vì tồn tại thao tác $\mathcal{O}(N)$, dùng riêng một trong hai cấu trúc đều không đủ tốt. Nếu kết hợp chúng, tức nhìn toàn cục như một danh sách liên kết nhưng mỗi nút chứa một mảng nhỏ có kích thước thích hợp, chẳng hạn 1000 hoặc 1500, ta nhận được một cấu trúc cân bằng hơn: đó là **danh sách liên kết theo khối**.

Với một biến nguyên ghi vị trí hiện tại, ta cần cấu trúc hỗ trợ ba thao tác:

1. chèn một đoạn thông tin tại vị trí chỉ định;
2. xóa một đoạn thông tin từ vị trí chỉ định;
3. lấy một đoạn thông tin từ vị trí chỉ định.

Cài đặt cơ bản gồm hai thao tác nội bộ:

- **định vị**: đi từ khối đầu tiên về sau cho tới khi tìm được khối chứa vị trí cần tìm, cùng vị trí tương đối trong khối đó;
- **tách khối**: tách một khối tại một vị trí thành hai khối.

Từ đó, thao tác chèn tìm vị trí, tách khối, rồi thêm các khối mới cho tới khi chèn xong. Thao tác xóa tìm đầu đoạn, tách khối đầu và khối cuối, rồi bỏ toàn bộ các nút nằm giữa. Thao tác lấy đoạn cũng định vị điểm bắt đầu, sau đó quét qua các khối cho tới khi đủ số ký tự.

Trong bản Word gốc có hai hình minh họa cho thao tác chèn và xóa. Hình chèn đánh dấu vị trí “bắt đầu chèn”, tách khối hiện tại rồi móc các khối mới vào giữa. Hình xóa đánh dấu đoạn giữa hai đường biên, tách hai đầu đoạn rồi bỏ các khối nằm trọn trong đoạn đó.

Một vấn đề còn lại là việc tách khối quá thường xuyên có thể tạo ra nhiều khối liên tiếp chứa rất ít dữ liệu, làm giảm hiệu quả. Vì vậy sau mỗi thao tác ta có thể gộp các khối liên tiếp quá nhỏ. Như vậy ta đã có một danh sách liên kết theo khối cơ bản đủ đáp ứng yêu cầu của bài.

2 Ứng dụng đơn giản

Danh sách liên kết theo khối có thể xem là một cách tối ưu hóa mô phỏng. Nó ghép hai cấu trúc không hoàn hảo thành một cấu trúc thực dụng hơn. Nhờ đặc điểm đó, với một số bài ta có thể mô phỏng trực tiếp một cách khá “cứng bức”.

2.1 NEERC 2003 Northern Subregion: Key Insertion

Có N binh sĩ đang tập đội hình và M vị trí từ trái sang phải, với $1 \leq N, M \leq 131072$. Mỗi lệnh có dạng $\text{Goto}(L, S)$. Nếu vị trí L trống thì binh sĩ S đứng vào đó. Nếu vị trí L đã có binh sĩ K , binh sĩ S đứng vào L , rồi tiếp tục thực hiện $\text{Goto}(L + 1, K)$. Mỗi binh sĩ nhận đúng một lệnh. Cần tìm trạng thái cuối cùng của đội hình; trong quá trình đó chỉ số vị trí có thể vượt quá M , nên trước hết phải xác định độ dài cuối cùng. Vị trí trống được in bằng 0.

Một cách hơi thô là nhìn thao tác chèn như sau: đẩy cả đoạn liên tiếp bắt đầu tại vị trí hiện tại sang phải một ô, rồi đặt phần tử mới vào chỗ đó. Tương đương, ta xóa một ô trống ngay sau đoạn liên tiếp ấy và chèn một phần tử mới ở vị trí hiện tại. Việc duy trì ô trống đầu tiên sau một vị trí là ứng dụng kinh

diễn của DSU, còn chèn xóa trên dãy lớn là điểm mạnh của danh sách liên kết theo khối. Do đó có thể ghép DSU với danh sách khối để giải bài.

Lời giải chính thống của bài tính tế hơn. Nó dùng DSU duy trì các khối vị trí không kề nhau, và dùng danh sách liên kết lưu các phần tử trong mỗi khối. Khi bình sĩ A được chèn vào một vị trí đã thuộc khối B , ta đặt A vào đầu danh sách của khối B . Nếu thao tác chèn làm một hoặc nhiều khối nối lại với nhau, danh sách của khối phía sau được nối vào trước danh sách của khối phía trước. Sau khi xử lý xong, duyệt từ khối cuối cùng về trước; với mỗi phần tử x , chèn nó vào ô trống thứ $L[x]$. Như vậy thu được dãy cuối cùng.

Chỉ riêng mô tả cách này đã khá phức tạp, chưa nói tới việc nghĩ ra trong phòng thi. Thực tế năm đó không đội nào giải được bài. Trong khi đó, với danh sách liên kết theo khối, cách làm mô phỏng gần như không cần nhiều suy nghĩ; có thể xem như một cách lấy điểm bằng mô phỏng mạnh hơn.

3 Phân tích hiệu năng: chọn kích thước khối

Trong trường hợp lý tưởng, giả sử kích thước mảng nhỏ trong mỗi nút là x . Khi đó có khoảng n/x khối. Thao tác định vị tốn $\mathcal{O}(n/x)$, còn phần ảnh hưởng phụ khi thêm hoặc xóa trong một khối tốn $\mathcal{O}(x)$. Hai chi phí cân bằng nhất khi

$$x \approx \sqrt{n},$$

và độ phức tạp của mỗi thao tác vào khoảng $\mathcal{O}(\sqrt{n})$. Trong thực tế n thay đổi liên tục, nhưng chọn x theo cỡ $\sqrt{N}$, với N là cận trên của số phần tử, vẫn giữ được hiệu năng xấp xỉ $\mathcal{O}(\sqrt{N})$ trong trường hợp xấu. Nói cách khác, dùng danh sách khối là theo đuổi sự cân bằng.

Bản gốc có hai biểu đồ thời gian: một cho 10 test của *NOI 2003 Editor*, một cho 38 test của *Key Insertion*. Các hình không được chép lại ở đây; nội dung quan trọng là quan sát của tác giả: kích thước khối lý thuyết không phải luôn chạy nhanh nhất, còn kích thước lớn hơn giá trị lý thuyết thường cho tốc độ tốt hơn.

Nguyên nhân đến từ cách cài đặt. Thao tác chèn và xóa đều phụ thuộc vào tách khối, nên việc tách diễn ra thường xuyên và đa số khối không đầy. Kích thước lưu trữ thực tế trong mỗi khối nhỏ hơn dung lượng tối đa. Vì vậy để kích thước thực tế gần $\sqrt{n}$, dung lượng tối đa nên lớn hơn $\sqrt{n}$. Mặt khác, vì hai khối kề nhau đều dưới nửa dung lượng sẽ được gộp, dung lượng tối đa cũng không cần lớn hơn $2\sqrt{n}$.

Với *Editor*, tác giả nhận thấy dung lượng khối khoảng 6000 mới nhanh nhất, lệch khá xa phân tích lý thuyết. Lý do là nhiều test chứa rất nhiều nhóm lệnh như:

```
MOVE x,  PREV,  NEXT,
GET 1,  GET 1,  GET 1.
```

Những nhóm này không thay đổi cấu trúc khối; với GET 1, chi phí lấy ký tự gần như không phụ thuộc kích thước khối. Nhưng trước khi lấy ký tự vẫn cần định vị, nên chúng thực chất làm tăng tỷ trọng thao tác định vị. Khối càng lớn thì số khối càng ít, định vị càng nhanh. Ngoài ra đề giới hạn tổng số INSERT và DELETE chỉ 4000, nên kích thước khối không ảnh hưởng mạnh bằng chi phí định vị.

Tác giả cũng viết một phiên bản dùng hai biến để ghi khối hiện tại và vị trí tương đối trong khối. Vì cài đặt dùng danh sách một chiều, thao tác PREV vẫn khó tránh khỏi định vị lại. Phiên bản này nhanh hơn, kích thước tối ưu giảm còn khoảng 5000, và khối càng lớn thì mức cải thiện càng nhỏ. Điều đó cho thấy một phần lớn lợi thế của khối lớn trong phiên bản đầu đến từ chi phí định vị. Nếu dùng danh sách hai chiều và xây một đối tượng gần giống iterator, khối nhỏ có thể nhanh hơn nữa, nhưng mã sẽ dài hơn. Tác giả không đưa iterator vào cấu trúc cơ bản vì nó không phải lúc nào cũng hữu ích, chẳng hạn *Key Insertion* không cần, còn việc duy trì iterator làm tăng độ dài mã và khả năng lỗi.

Với *Key Insertion*, thời gian khi kích thước khối từ 400 tới 1200 gần như nhau, tốt nhất quanh 800, tức nằm giữa $\sqrt{n}$ và $2\sqrt{n}$. Tóm lại, hiệu năng danh sách khối phụ thuộc dữ liệu thực tế, nhưng thường nên chọn dung lượng khối lớn hơn $\sqrt{n}$ một chút để lượng dữ liệu thực trong mỗi khối gần $\sqrt{n}$. Đây vẫn là nguyên tắc cân bằng.

Do tách khối thường xuyên kéo theo cấp phát và giải phóng bộ nhớ, toàn bộ các chương trình của tác giả dùng mảng tĩnh và tự duy trì danh sách nút rảnh.

4 Mở rộng danh sách liên kết theo khối

Xét một bài phức tạp hơn.

4.1 NOI 2005: Sequence

Cần duy trì một dãy số và hỗ trợ sáu thao tác:

Số	Thao tác	Ý nghĩa
1	INSERT <i>posi tot c1 ... ctot</i>	chèn <i>tot</i> số sau vị trí <i>posi</i>
2	DELETE <i>posi tot</i>	xóa <i>tot</i> số từ vị trí <i>posi</i>
3	MAKE-SAME <i>posi tot c</i>	gán đoạn dài <i>tot</i> thành <i>c</i>
4	REVERSE <i>posi tot</i>	đảo đoạn dài <i>tot</i>
5	GET-SUM <i>posi tot</i>	hỏi tổng đoạn dài <i>tot</i>
6	MAX-SUM	hỏi tổng lớn nhất của một đoạn con liên tiếp

Tại mọi thời điểm dãy có ít nhất một số, input luôn hợp lệ. Với 50% dữ liệu, dãy có nhiều nhất 30000 số; với toàn bộ dữ liệu, dãy có nhiều nhất 500000 số. Mỗi số nằm trong $[-1000, 1000]$, số thao tác $M \leq 20000$, tổng số phần tử được chèn không quá 4000000, input không quá 20 MB.

Mô phỏng trực tiếp thì dễ nhưng không đủ nhanh, và bài cũng không gợi ra một thuật toán hoàn toàn khác. Đây là lúc danh sách liên kết theo khối lại hữu dụng. So với bài trước, ta cần thêm các thao tác:

1. đảo một đoạn con;
2. gán cả một đoạn con thành cùng một giá trị;
3. tính tổng một đoạn liên tiếp;
4. tính tổng đoạn con liên tiếp lớn nhất của toàn dãy.

Ưu điểm của danh sách khối là xử lý nguyên khối. Vì vậy mỗi khối cần lưu thêm thông tin để các thao tác trên đoạn có thể bỏ qua các khối nằm trọn trong đoạn. Để hỏi tổng đoạn con lớn nhất, mỗi khối lưu tổng của khối, tổng đoạn con lớn nhất bên trong khối, và tổng lớn nhất của đoạn con chứa đầu trái hoặc đầu phải của khối. Để gán nhanh cả khối thành một giá trị, dùng một cờ đánh dấu khối đồng nhất và một biến lưu giá trị đó. Tương tự, mỗi khối có thêm cờ đảo để biểu diễn trạng thái đang bị lật.

Lúc này mới thấy lợi ích của cách cài đặt trước đó: chèn và xóa không cần xét quá nhiều thông tin mới, còn REVERSE và MAKE-SAME có thể dùng cùng khuôn xử lý. Việc cập nhật thông tin tập trung chủ yếu vào thao tác tách khối, nên chỉ cần viết phần đó thật cẩn thận.

Một mẹo nhỏ là lưu hai giá trị biên trong mảng `sidemax[2]`: `sidemax[0]` là tổng lớn nhất của đoạn con chứa đầu trái của khối khi chưa xét trạng thái đảo, còn `sidemax[1]` tương tự cho đầu phải. Khi đó `sidemax[reversed]` chính là giá trị thực tế chứa đầu trái, và `sidemax[!reversed]` là giá trị thực tế chứa đầu phải.

5 Các ứng dụng phức tạp hơn

5.1 CERC 2007: Sort

Trong một xưởng có N linh kiện xếp thành hàng, $1 \leq N \leq 100000$, biết chiều cao của từng linh kiện. Cần sắp xếp chúng tăng dần, nhưng chỉ được dùng thao tác sau: tìm vị trí P_1 của linh kiện thấp nhất và đảo đoạn $[1, P_1]$, rồi tìm vị trí P_2 của linh kiện thấp thứ hai và đảo $[2, P_2]$, cứ tiếp tục như vậy. Chương trình phải in ra $P_1, P_2, P_3, \dots$. Nếu nhiều linh kiện có cùng chiều cao, xử lý linh kiện đứng trước trong dãy ban đầu trước.

Thoạt nhìn khó tìm ngay một thuật toán đặc biệt bằng cây cân bằng, cây đoạn hay heap. Vậy cứ mô phỏng. Ta vẫn dùng danh sách khối: vì cần đảo đoạn nên thêm cờ đảo; vì mỗi bước cần tìm phần tử nhỏ nhất của dãy hiện tại nên mỗi khối lưu thêm giá trị nhỏ nhất của khối. Ở mỗi bước, tìm khối chứa giá trị nhỏ nhất cần xử lý, tìm vị trí cụ thể trong khối, đảo đoạn tiền tố, xóa phần tử đầu, rồi lặp lại.

Vấn đề duy nhất là các giá trị bằng nhau và yêu cầu ưu tiên phần tử xuất hiện sớm hơn trong dãy gốc. Có thể khử vấn đề này bằng cách thay giá trị của mỗi phần tử bằng thứ tự xử lý của nó trong dãy ban đầu. Khi đó không còn trùng giá trị, phần còn lại là cài đặt danh sách khối.

5.2 NOI 2007: Necklace

Một hệ thống cho phép khách hàng tự thiết kế dây chuyền. Dây chuyền có N hạt, $1 \leq N \leq 500000$, mỗi hạt có màu thuộc $1, 2, \dots, c$. Dây được cố định trên một tấm phẳng; vị trí đánh dấu là 1, các vị trí còn lại được đánh số theo chiều kim đồng hồ.

Hệ thống cần hỗ trợ:

- R k : quay dây chuyền theo chiều kim đồng hồ k vị trí;
- F: lật tấm phẳng theo trục đối xứng cho sẵn;
- S i j : đổi chỗ hai hạt ở vị trí i, j ;
- P i j x : tô đoạn từ i tới j theo chiều kim đồng hồ thành màu x ;
- C: hỏi toàn bộ dây chuyền hiện có bao nhiêu đoạn màu liên tiếp;
- CS i j : hỏi đoạn từ i tới j theo chiều kim đồng hồ gồm bao nhiêu đoạn màu liên tiếp.

Thông tin cần duy trì lúc này là số đoạn màu. Để thuận tiện, mỗi khối còn lưu màu hai đầu. Lật và quay có thể quy về thao tác đảo trước đó; truy vấn C có thể dùng CS. Vì vậy cần thêm hai thao tác chính: Swap và CountSegment. Swap chỉ cần tìm hai phần tử cần đổi; nếu chúng khác nhau thì đổi rồi cập nhật thông tin của các khối liên quan. CountSegment giống truy vấn tổng đoạn: khối đầu và cuối xử lý riêng, các khối nằm giữa dùng thông tin đã lưu.

6 Độ phức tạp cài đặt và tốc độ

Với *CERC 2007 Sort*, mã của tác giả dài 3.44 KB nhưng không nhanh. Do không biết thời hạn của kỳ thi, tác giả so sánh với chương trình chuẩn dài 4.94 KB, chứa nhiều chú thích và khoảng trắng: chương trình chuẩn chạy 0.61 giây, còn chương trình danh sách khối chạy 3.63 giây.

Với *NOI 2007 Necklace*, mã danh sách khối dài 6.34 KB và chỉ qua 7 test. Nếu nhận ra thao tác lật và quay là những phép biến đổi tọa độ “đánh lừa”, thêm tối ưu có thể qua 9 test. Tuy nhiên điều này cũng cho thấy với người bình thường, viết danh sách khối đủ nhanh không dễ. Nếu đã nghĩ đến biến đổi tọa độ để xử lý lật và quay, ta cũng có thể nghĩ tới cây đoạn, đơn giản và nhanh hơn. Chênh lệch giữa $\mathcal{O}(\sqrt{n})$ và

$\mathcal{O}(\log n)$ là hàng chục lần, hằng số của danh sách khối lại lớn hơn cây đoạn. Mã Pascal cây đoạn của Yu Linyun chỉ dài 4.12 KB và nhanh hơn rất nhiều.

7 Kết luận

Ban đầu tác giả muốn giới thiệu danh sách liên kết theo khối như một cách lấy điểm bằng mô phỏng, vì mô phỏng giúp tiết kiệm thời gian suy nghĩ. Nhưng cuối cùng hiệu quả lấy điểm không tốt như mong đợi. Trước hết, mã danh sách khối thường khá dài và dễ sai. Các bài hiện nay hay kết hợp nhiều cấu trúc dữ liệu, như *Key Insertion*; yêu cầu biến đổi trước khi mô phỏng, như *Sort*; hoặc cần cấu trúc có nhiều chức năng, như *Sequence* và *Necklace*. Những tình huống đó làm việc đảm bảo đúng rất khó.

Thứ hai, ngay cả khi viết nhanh và đúng, hiệu năng vẫn không chắc chắn. Chọn kích thước khối không tốt có thể vượt thời gian; đôi khi chọn gần tối ưu vẫn có thể không đủ, vì trên dữ liệu lớn $\mathcal{O}(\sqrt{n})$ và $\mathcal{O}(\log n)$, thậm chí $\mathcal{O}(1)$ trong một số bài, cách nhau rất xa.

Dẫu vậy, danh sách khối không hề vô dụng. Nó kết hợp mảng và danh sách liên kết, hai cấu trúc cơ bản đều có khuyết điểm, rồi xử lý theo khối để cân bằng hai loại thao tác. Tư tưởng ghép các phần tử cơ bản, thậm chí các phần tử cơ bản không thật nhanh, để tạo thành kết quả hiệu quả hơn vẫn rất đáng học.

Hơn nữa, danh sách khối dễ mở rộng. Với thao tác trên dãy như gán đồng loạt, đảo đoạn, hỏi tổng, duy trì giá trị nhỏ nhất, v.v., ta thường có thể đạt $\mathcal{O}(\sqrt{n})$. Nếu duy trì toàn bộ danh sách khối theo thứ tự, nó còn có thể mô phỏng một số thao tác của cây cân bằng, vì cây cân bằng cũng có thể được nhìn như một cách duy trì dãy.

Do mỗi khối chỉ lưu một lượng thông tin phụ, mức sử dụng bộ nhớ khá tốt. Cùng là mô phỏng, splay phải lưu toàn bộ thông tin phụ ở từng nút; tốc độ nhanh hơn nhưng tốn bộ nhớ hơn.

Trong đời sống, tư tưởng danh sách khối cũng xuất hiện thường xuyên. Hệ thống tập tin FAT truyền thống chia các sector đĩa thành cluster, rồi dùng bảng FAT (*File Allocation Table*) ghi trạng thái của từng cluster: có hỏng không, có đang được dùng không, và nếu được dùng thì cluster kế tiếp là gì. Cách tổ chức đó rất gần với danh sách liên kết theo khối.

Vì danh sách khối tiết kiệm không gian và cấu trúc khối dễ kết hợp với bộ đệm, Vim cũng dùng cấu trúc tương tự cho lưu trữ trong bộ nhớ và bộ đệm trên đĩa. Nếu dùng splay để viết trình soạn thảo, cấu trúc sẽ phức tạp và trừu tượng hơn nhiều; cũng khó tận dụng bộ đệm và khó khôi phục cây từ dữ liệu đệm sau sự cố như mất điện hoặc chương trình sụp đổ.

Ngoài ra, thư viện `<ext/rope>` của g++ đã có một mẫu danh sách khối cơ bản: `__gnu_cxx::rope<T, Alloc>`. Vì vậy lập trình viên C++ có thể dùng ngay một cấu trúc có sẵn.

Lời cảm ơn

Tác giả cảm ơn thầy Liu Rujia đã góp ý về đề tài và nội dung bài viết; cô Hou Xiaojing đã góp ý chỉnh sửa; Chen Yanhui đã cung cấp mã cho *NOI 2005 Sequence*, giúp tác giả học thêm kinh nghiệm viết danh sách khối; Gao Yihan đã giải thích cách ước lượng độ phức tạp; và Zhou Mengyu đã giới thiệu mẫu danh sách khối trong thư viện `<ext/rope>`.

Tài liệu tham khảo và ghi chú

- Bài viết năm 2005 của Long Fan về ứng dụng của thứ tự.
- Bài viết năm 2007 của Yu Jiangwei về xử lý tốt các bài thống kê động.

- Lời giải *NOI 2007* của Yu Linyun.
- Dữ liệu và chương trình chuẩn của *Key Insertion* tại <http://neerc.ifmo.ru/past/2003.html>.
- Dữ liệu và chương trình chuẩn của *Sort* tại <http://contest.felk.cvut.cz/07cerc/>.
- Giới thiệu về Vim của Bram Moolenaar tại <http://www.free-soft.org/FSM/english/issue01/vim.html>.
- Tài liệu về rope tại <http://www.sgi.com/tech/stl/> và <http://www.sgi.com/tech/stl/Rope.html>.

Một số ghi chú nhỏ trong bản gốc: còn có những cách cài đặt nhanh hơn, nhưng cách mô phỏng bằng danh sách khối giúp giảm bớt nhánh xử lý và ít lỗi hơn; bản dịch đề *Sequence* lấy từ bài trại đông năm 2005 của Long Fan; máy thử nghiệm của tác giả dùng CPU 1.8 GHz hai nhân và 1 GB RAM; mã thử nghiệm nằm trong phần phụ lục của tài liệu gốc. Tác giả cũng ghi rằng có thể dùng kỹ thuật rời rạc hóa thời gian thực cho một số dữ liệu của *Editor*; một phiên bản danh sách khối phức tạp hơn của tác giả qua 8 test *Necklace* và hơi quá thời gian ở test thứ 9; với *Happybirthday* của NOI 2006, mã danh sách khối ngắn hơn một chút so với mã SBT của tác giả và qua được 8 test. Nếu dùng splay, có thể giảm độ phức tạp mô phỏng xuống $\mathcal{O}(\log n)$, đánh đổi bằng bộ nhớ lớn hơn.